Dated ETF Allocation Disclosures: A Relational Design and Data-Access Layer*

Taimoor Shahzad
University of Victoria
Victoria, Canada

M. Shahmeer Ather Kakwan
University of Victoria
Victoria, Canada

Mason Li
University of Victoria
Victoria, Canada

Abstract

Exchange-traded fund (ETF) providers publish dated sector and geographic allocation percentages that are difficult to compare across funds and time. We present a relational design for dated ETF allocation disclosures that separates a customer's holdings from a provider's disclosure, scopes category identity by classification or capture context, and enforces integrity through STRICT tables, foreign keys, triggers and validated transactions. A data-access layer exposes parameterized functions for search, detail, portfolio and create/update/delete operations over a fully synthetic demonstration dataset, and a Flask web interface presents the same operations to users. The suite passes 74 tests and the full demonstration is reproducible from a clean checkout through a single quickstart command. The project repository is https://github.com/Pyrex111/CSC501_Data_Modeling_Project.

1 Design and Implementation

1.1 Relational model and integrity

The revised ER model maps to twelve relations with thirteen foreign keys. `Holding` represents the dated Portfolio–ETF association; `AllocationEntry` represents a Snapshot–Category association and its reported weight. Holding and allocation observation dates remain independent. Each relation uses an autogenerated integer `Id`, while mandatory business identifiers are preserved through uniqueness and non-null constraints. These include the customer's reference number, the customer's portfolio reference, and a company's registration number within its issuing jurisdiction.

ETF identity is supplied by its identifier records. ISIN uniqueness excludes exchange context; ticker uniqueness includes a required exchange. Partial unique indexes protect these alternatives and known classification editions without inventing missing versions. A captured source is identified by URL and retrieval time. Category identity is scoped by dimension and context: a verified scheme edition, or an individual captured allocation block when classification is unresolved. Identical labels across contexts do not establish equivalent meaning.

SQLite STRICT tables enforce scalar types; named checks constrain dates, fractions and allowed values. Foreign-key enforcement is enabled per connection [1, 2]. Four triggers prevent category and snapshot dimensions from disagreeing, including after parent edits. A validated transaction additionally checks required child records, category contexts and complete-disclosure totals within a provisional 0.005 rounding tolerance, pending justification from genuine source precision. These transaction rules require the supplied application boundary; they are not database-level deferred assertions.

The schema passed 33 behavioral tests using explicitly synthetic fixtures, including duplicate-key rejection, invalid dates/weights, dimension conflicts, context isolation and rollback of invalid final states. These results validate the schema contribution. Eleven additional integration regression tests passed after correcting batch-update rollback, dated portfolio selection and missing classification/source outputs. The quickstart now preserves existing data unless reset is explicitly requested, and both setup paths use the validated seed loader. The full suite now passes 74 tests: the 33 schema tests, the 11 integration regressions and 30 UI-route tests. The Flask interface is integrated over the same parameterized operations. The retained demo source files remain text placeholders, so file existence does not verify provider observations.

1.2 Data access, queries and reproducibility

The demonstration is fully synthetic: every customer, provider, source identity, captured file, classification example and weight is fictional. Source URLs use the reserved `example.invalid` domain and are not real addresses. The same dataset is produced by the executable, parameterized loader `seed_data.py` and the human-readable `seed.sql`; their logical dumps are identical. All five Complete snapshots sum to exactly 1.0, while the twelve tables retain the agreed row counts (three ETFs, six identifiers, thirteen categories, seventeen entries).

The six SELECT/JOIN queries are best read through representative examples rather than as a catalogue. The geographic query joins entries to their snapshot, dimension, ticker and source and returns the observation date on every row, so the 2026-05-31 and 2026-06-30 figures are never silently summed as if measured together. The per-snapshot aggregate returns the category count and rounded total, giving a direct check that each disclosure is complete and closed. The fifth query is deliberately a *diagnostic candidate listing*: it pairs historical holdings with candidate geographic rows and multiplies them, but it does not select an applicable disclosure or produce a validated portfolio exposure. A genuine look-through would still require an explicit holding date, disclosure selection, coverage/basis checks and evidence that the category meanings are compatible.

Create, update and delete operations preserve the same integrity boundary. Every user value is bound through placeholders, and each write runs inside the validated transaction

*Project repository: https://github.com/Pyrex111/CSC501_Data_Modeling_Project

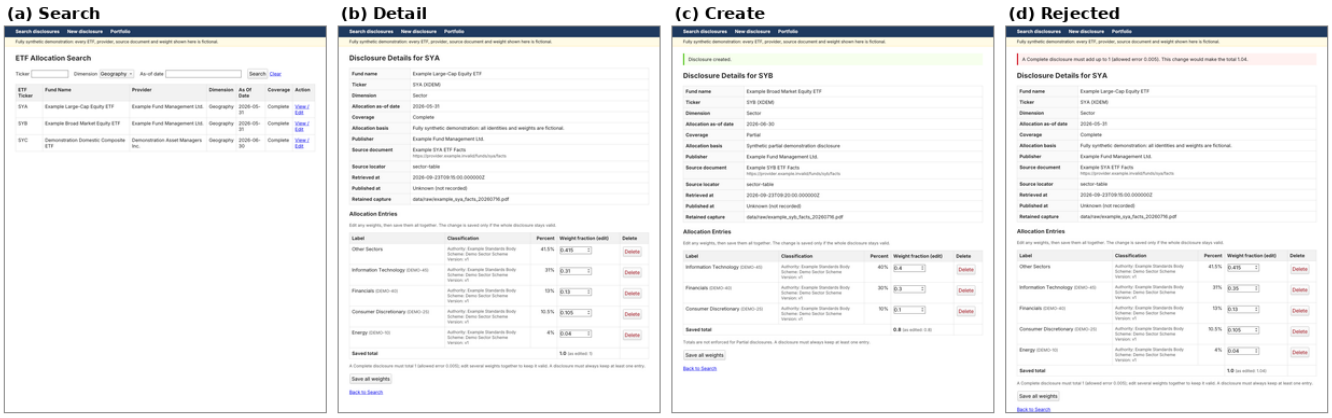


Figure 1: Web interface: search results, disclosure detail with source evidence, the create form, and a rejected update (fully synthetic data).

that verifies the whole database before COMMIT. Creating a disclosure inserts the snapshot and all its entries as one atomic unit, and an invalid total is rolled back rather than partially saved. A single weight edit, or a batch of edits, is rejected when it would break a Complete total; a missing identifier inside a batch now rolls back the earlier valid changes. Deletion is not restricted by type: the same function refuses removing the last entry of a snapshot, or any edit that would leave a Complete total off 1 ± 0.005, so a successful deletion is shown on a Partial disclosure. Failures surface as `ValidationError`, `NotFoundError` or `BusinessRuleError`, which the UI can render as friendly messages.

A single `quickstart.py` makes the demonstration reproducible from a clean checkout. It reuses an existing database and only replaces it after an explicit `--reset`; the replacement is built in a temporary location and atomically published, and a failed seed leaves the previous file intact. The full suite passes 74 tests—33 schema tests, 11 integration regressions and 30 UI-route tests—and the quickstart runs all six queries.

1.3 Web Interface and Validation

The Flask interface has three layers: routes (`src/ui`), business logic (`src/biz`) and the data-access layer, so routes contain no SQL. Users search disclosures by ticker, dimension and as-of date; open a disclosure to see its basis, coverage, classification authority, scheme and version (or "not recorded"), source URL, locator, retrieval and publication dates and retained file; create a disclosure with its entries; edit several weights together; delete an entry; and view a portfolio with the holding date shown apart from each ETF's allocation dates.

Input is checked in the browser, the business layer (real dates, fractions in $[0, 1]$, no duplicate categories, at least one entry), the DAL (parameterized SQL) and the database, where `validated_transaction` rejects any final state that breaks a rule, such as a Complete disclosure not summing to 1 ± 0.005. Rejected changes roll back and show a plain message and the user's typed values, never a server error.

We ran 30 route tests (74 with the schema suite, all passing) and exercised the running application; rejected writes left the database unchanged. Complete disclosures only accept coherent multi-entry edits, so deletion is demonstrated on a Partial synthetic disclosure.

References

[1] SQLite Development Team. [n. d.]. *SQLite Foreign Key Support*. https://www.sqlite.org/foreignkeys.html
[2] SQLite Development Team. [n. d.]. *STRICT Tables*. https://www.sqlite.org/stricttables.html

2 Relations and constraints

Every relation has an autogenerated `Id` INTEGER primary key. Foreign-key columns are INTEGER; `WeightFraction` is REAL; other columns are TEXT. Tables are STRICT. All columns are NOT NULL except those marked `?`; required text is nonempty, and identity text has no surrounding spaces. All FKs reference `Id`, with UPDATE/DELETE RESTRICT. **UK** denotes uniqueness; conditional UKs apply only to their stated rows.

Customer
`Id, CustomerNumber, GivenName, FamilyName, Email?`
UK: CustomerNumber. Names and Email are not unique.

Portfolio
`Id, CustomerId, PortfolioReference, PortfolioName, BaseCurrency`
FK: CustomerId → Customer. UK: (CustomerId, PortfolioReference).

Holding
`Id, PortfolioId, ETFId, WeightFraction, HoldingAsOfDate`
FKs: PortfolioId → Portfolio; ETFId → ETF. UK: (PortfolioId, ETFId, HoldingAsOfDate).

ETFProvider
`Id, LegalName, RegistrationNumber, RegistrationJurisdiction, WebsiteUrl?`
UK: (RegistrationJurisdiction, RegistrationNumber). LegalName is descriptive, not unique.

ETF
`Id, ETFProviderId, FundName, DomicileCountryCode, BaseCurrency, InceptionDate?`
FK: ETFProviderId → ETFProvider. Business identity is supplied by ETFIdentifier; FundName is not unique. Each saved ETF requires at least one identifier (transaction rule).

ETFIdentifier
`Id, ETFId, IdentifierScheme, IdentifierValue, ExchangeContext?`
FK: ETFId → ETF. IdentifierScheme ∈ {ISIN, Ticker}. Values are uppercase and nonempty. ISIN: UK (IdentifierScheme, IdentifierValue), ExchangeContext is NULL; value has 12 uppercase alphanumeric characters, two initial letters and a final digit. Ticker: UK (IdentifierScheme, IdentifierValue, ExchangeContext); exchange is required, uppercase and nonempty. ISIN checksum is not checked.

Common scalar rules. Fractions are in $[0, 1]$. Currency codes have three uppercase letters and domicile codes two. Dates are valid `YYYY-MM-DD`; source timestamps are UTC `YYYY-MM-DDTHH:MM:SS.ffffffZ`. Optional text is NULL when unknown and nonempty when present. Code formats do not verify external registry membership.

SourceDocument
`Id, ETFProviderId, Url, RetrievedAt, PublishedAt?, CapturedFileRef, DocumentTitle?`
FK: ETFProviderId → ETFProvider. UK: (Url, RetrievedAt). CapturedFileRef identifies a retained capture; the transaction validator checks file existence within the supplied project root.

AllocationDimension
`Id, DimensionName`
UK: DimensionName; DimensionName ∈ {Sector, Geography}.

ClassificationScheme
`Id, SchemeAuthority, SchemeName, Version?, DefinitionUrl?`
Conditional UK: (SchemeAuthority, SchemeName, Version) WHERE Version IS NOT NULL. Unknown editions retain NULL Version and no asserted complete business key.

AllocationCategory
`Id, AllocationDimensionId, ClassificationSchemeId?, Label, Code?, ContextKey`
FKs: AllocationDimensionId → AllocationDimension; ClassificationSchemeId → ClassificationScheme (optional). UK: (AllocationDimensionId, ContextKey, Label). ContextKey is generated from known authority/name/version or from captured URL/retrieval time/block locator/dimension. Unresolved contexts are local to the disclosure; matching labels/codes do not prove equivalence.

AllocationSnapshot
`Id, ETFId, AllocationDimensionId, SourceDocumentId, AllocationAsOfDate, BasisNote, CoverageStatus, SourceLocator`
FKs: ETFId → ETF; AllocationDimensionId → AllocationDimension; SourceDocumentId → SourceDocument. UK: (ETFId, AllocationDimensionId, AllocationAsOfDate, SourceDocumentId, SourceLocator). CoverageStatus ∈ {Complete, Partial, Unknown}. SourceLocator distinguishes dated allocation blocks/bases within a capture.

AllocationEntry
`Id, AllocationSnapshotId, AllocationCategoryId, WeightFraction`
FKs: AllocationSnapshotId → AllocationSnapshot; AllocationCategoryId → AllocationCategory. UK: (AllocationSnapshotId, AllocationCategoryId).

Cross-relation rules. SQL triggers require matching category/snapshot dimensions on entry writes and parent dimension edits. Before COMMIT, the transaction validator requires an entry per saved snapshot, validates used category contexts and checks Complete totals within 0.005 of 1 (inclusive, configurable). Partial/Unknown totals and sampled holding totals need not equal 1. Parent minimums, contexts, totals and file checks require the application validator; raw SQL COMMIT can bypass them.